

Problem Statement and Goals

Tandem

Team 11, Tandem
Justin Kwinecki
Zayed Umer
William Pol
Manav Patel
Musab Mondal

Table 1 records the changes to this document.

Table 1: Revision History

Date	Developer(s)	Change
Sept 22, 2026	Justin Kwinecki	Initial problem statement and goals
Sept 27, 2026	Zayed Umer	Revise problem statement and goals; see disclosure appendix
Sept 27, 2026	Zayed Umer	Cut repetition and clarify wording
Sept 27, 2026	Zayed Umer	Revise the problem section from review

1 Problem Statement

Tandem aims to reduce repeated prompt processing when several software agents use the same background information.

1.1 Problem

Modern applications increasingly use several agents that work from the same underlying context. Each agent submits its own request, but those requests share a long prefix: the identical opening of their prompts. Before a model can generate a response, it processes the prompt in a phase called prefill. When each request prefills that shared prefix on its own, the same work is repeated.

As more agents run at the same time, the repeated prefill delays responses, wastes computing capacity, and limits how many requests can be served. Existing serving software can reuse a prefix only after it has already been processed [1]. The repeated work remains when requests arrive together, or while their shared prefix is still being processed.

The problem is to reuse a shared prefix during prefill across simultaneous requests, so that work is done once, while each request’s model output stays the same.

1.2 Scope

Tandem targets concurrent text-generation requests that use the same model and share an identical prefix followed by different instructions. Requests may arrive together or a short time apart. Requests without shared context serve as a compatibility check.

Evaluation will compare identical workloads, models, hardware, and generation settings with and without Tandem’s changes. The baseline will retain the serving engine’s existing prefix reuse features. The project excludes model training, agent orchestration, document retrieval, application interfaces, and operation across multiple servers. Reuse of similar but differently worded context is also outside the scope.

The proof of concept will test whether the baseline repeats work and whether Tandem preserves model results. If an optimization is infeasible, the fallback is a tool that measures and reports repeated prompt processing. The team would revise the goals with the teaching assistant (TA) and instructor. The [Development Plan](#) describes that evaluation and fallback.

1.3 Inputs and Outputs

The inputs are:

- Requests, each with an identifier, prompt, and generation settings such as the maximum response length and sampling controls.
- A trained model and the encoding rules that convert text into tokens, the units the model processes.
- For evaluation, the requests’ arrival times and the serving configuration.

The outputs are:

- Generated text for each accepted request, with its identifier and a completion status or failure indication.
- Evaluation results: response timing, completed requests per second, prompt tokens processed, and agreement with baseline outputs. Results include the workload and configuration needed to reproduce them.

1.4 Stakeholders

- **Application developers, including independent developers:** need faster responses without changing their agents’ tasks or results.
- **Serving-engine maintainers:** need changes they can integrate and maintain, supported by correctness and compatibility tests.
- **Infrastructure operators and deploying organizations:** need to serve more requests within their hardware capacity and budget.
- **Application end users:** need timely responses that are unaffected by other agents’ private instructions.
- **The capstone team, TA, and instructor:** need a feasible scope and reproducible evidence that the project meets its goals.

1.5 Environment

Developers and operators will run Tandem on a local or remote Linux server with a supported graphics processing unit (GPU) and enough memory for the selected model and requests. They supply the model and compatible serving software. Applications submit requests through a programmatic interface; end users interact with those applications.

Initial evaluation uses one server and one model configuration at a time. The Development Plan covers developer tools and laboratory hardware.

2 Goals

These goals will be refined into acceptance criteria in the Software Requirements Specification and test procedures in the Verification and Validation Plan. Performance comparisons use the baseline in Section 1.2, repeated runs, and reported variation. Improvements must exceed that variation.

- G1. **Reduce repeated prompt processing.** Process fewer prompt tokens than the baseline on shared-prefix workloads. Count tokens actually processed, including recomputation. Compare simultaneous and staggered arrivals with previously processed and unseen prefixes.
- G2. **Reduce response delay.** Lower the median and 95th-percentile time to first token: the time from submitting a request to receiving its first generated token, including queuing.
- G3. **Increase serving capacity.** Complete more requests per second on the same hardware with matched response lengths. Report response delays alongside throughput.
- G4. **Preserve model behaviour and request independence.** Match baseline output scores within a documented numerical tolerance under controlled settings, including multi-token generation. Verify that changing one agent’s private instructions leaves another agent’s output scores unchanged within that tolerance.
- G5. **Preserve ordinary serving.** Continue to handle individual requests and requests without shared context. Pass the baseline’s functional checks and measure any added delay or loss of throughput.

Evaluation will report improvements, regressions, and workloads with no measurable change.

3 Stretch Goals

After meeting the core goals, the team may pursue:

- **Support deeper branching.** Extend correctness and performance evaluation to prompts with several shared branching points, including branches introduced during an ongoing interaction.
- **Support more configurations.** Repeat the evaluation on additional models or GPUs to test whether the benefits persist.
- **Reduce memory demand.** Use less peak GPU memory than the matched baseline for shared-context workloads and measure whether this permits more concurrent requests within the same memory limit.

4 Custom Documents (CDs)

The Development Plan draft proposes these custom documents, subject to instructor approval:

- **Fall: Machine Learning Model Report.** Justify the selected model, evaluation prompts, and correctness checks. Discuss model and data limitations, and explain why training procedures do not apply.

- **Winter: Performance Report.** Analyze repeated processing, response delay, throughput, and resource use. Record configurations, workloads, repeated trials, variation, and failure cases. The Verification and Validation Report will reference this analysis when assessing whether requirements were met.

References

- [1] vLLM contributors. Automatic prefix caching. https://docs.vllm.ai/en/latest/features/automatic_prefix_caching/. Accessed September 27, 2026.

Appendix — Generative AI Use Disclosure

OpenAI Codex revised all sections using Justin Kwinecki’s initial draft and proposed additions, the Development Plan draft, repository checklists, and course outline. It checked the cited prefix-caching documentation, edited for clarity and concision, and built and inspected the PDF. No performance experiments were run or novelty established.

Zayed Umer supplied reflection notes about collaborative brainstorming, the difficulty of finding a novel idea, and the motivation to improve memory use even modestly. Codex edited those notes and connected them to the proposed scope. The team remains responsible for reviewing the wording, claims, and scope before submission.

Appendix — Reflection

1. What went well while writing this deliverable?

We agreed early that we wanted to work on inference. That shared interest made brainstorming collaborative: we could build on each other’s ideas and focus on which problem to tackle.

2. What pain points did you experience during this deliverable, and how did you resolve them?

Finding a novel idea was harder than choosing the topic. With so much inference research underway, we struggled to tell whether an idea addressed an open problem. That uncertainty remains. We plan to review existing work and measure the selected serving engine before committing to an optimization. The proof of concept will test whether our proposed contribution is needed.

3. How did you and your team adjust the scope of your goals to ensure they are suitable for a Capstone project?

We wanted to improve inference resource use, especially memory, even if the gain was modest. The proposed scope turns that motivation into one workload we can measure: requests sharing the same context. Using existing models lets us focus on serving-system design, correctness, and performance. Memory savings and broader model support remain stretch goals. This gives us a bounded implementation to evaluate within the course, with success judged by reproducible gains.